Scientific Calculator Application: A DevOps CI/CD Pipeline Report

Shreyas Gangurde (MT2025117)

March 7, 2026

Contents

1 Introduction **2**

2 DevOps Automation Workflow **2**

3 Jenkins Pipeline Architecture (`Jenkinsfile` Explanation) **2**
3.1 Stage 1: Build (CMake) 3
3.2 Stage 2: Unit Tests 3
3.3 Stage 3: Integration Tests 3
3.4 Stage 4: Build Docker Image 3
3.5 Stage 5: Push Docker Image 4
3.6 Stage 6: Deploy (Ansible) 4
3.7 Post-Build Actions 4

4 Pipeline Execution and Observability **5**

5 Test Results and Observations **9**
5.1 Unit Test Results (Google Test) 9
5.2 Integration Test Results (Expect Automation) 9

6 Application Showcase **9**

7 Links and References **10**

1 Introduction

This report documents the Continuous Integration and Continuous Deployment (CI/CD) pipeline built for a Command-Line Interface (CLI) Scientific Calculator written in C++. The core focus of this project is automating the complete lifecycle from code compilation to web-based deployment.

The pipeline automates:

- Source code compilation utilizing CMake.
- Rigorous Unit and Integration Testing.
- Containerization of the application via Docker.
- Automated deployment to a server using Ansible.
- Serving the interactive C++ terminal application natively over a web browser via Gotty.

2 DevOps Automation Workflow

The following architecture visually tracks the flow of code from the developer's laptop to our live server environment.

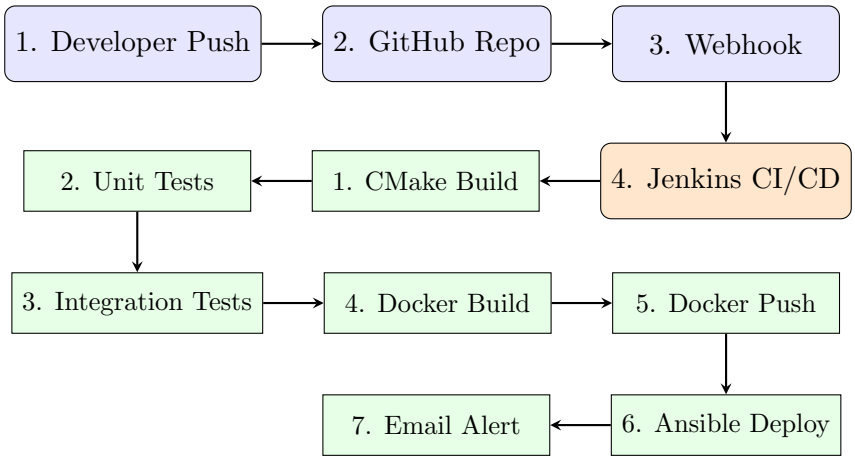

Figure 1: DevOps Automation Workflow and Jenkins Pipeline Architecture mapping code from branch check-in to automated remote deployment.

3 Jenkins Pipeline Architecture (`Jenkinsfile` Explanation)

The backbone of this automated process is a declarative Jenkins Pipeline. Initially, this project was started as a Freestyle project in Jenkins to serve as a Proof of Concept (PoC). Once the core deployment and testing concepts were validated, the CI/CD structure was migrated to a robust declarative Pipeline to manage the complex, multi-stage lifecycle. The older Freestyle builds and the newer Pipeline builds have been appropriately renamed in the Jenkins dashboard to clearly signify their purpose.

Below is a detailed breakdown of each stage executed sequentially whenever new code is merged into the development branch.

3.1 Stage 1: Build (CMake)

This stage is responsible for generating the Makefiles and compiling the C++ source code into an executable binary.

```
mkdir -p build && cd build
cmake ..
make
```

Explanation: We create an isolated `build` directory. `cmake ..` reads the root file `CMakeLists.txt` to locate source files and external dependencies (like Google Test). `make` compiles the actual `./calculator` binary alongside the testing executables.

3.2 Stage 2: Unit Tests

This stage isolates and validates the core mathematical logic directly.

```
cd build
ctest --output-on-failure -E "^IntegrationTest$"
```

Explanation: Using CTest (the test driver program for CMake), Jenkins triggers the isolated Google Test suite (`test_calculator.cpp`). This rigorously checks fundamental operations like square roots of negative numbers, factorial constraints, and natural logarithms without invoking the User Interface layer.

3.3 Stage 3: Integration Tests

This stage validates the interactive ‘ncurses‘ terminal interface exactly as a user would experience it.

```
export TERM=xterm-256color
ctest --output-on-failure -R "^IntegrationTest$"
```

Explanation: Because the application actively reads keyboard arrows, we use the `expect` automation tool. We explicitly set the `TERM` environment variable so the CI server behaves like a real terminal. The Expect scripts mock pressing keyboard keys to navigate the calculator’s menu and input numbers, asserting that the correct mathematical answers print directly to the screen.

3.4 Stage 4: Build Docker Image

Instead of deploying raw binaries, the application is packaged into a standardized container.

```
DOCKER_BUILDKIT=1 docker build -t shreyasg28/sci-calc .
```

Explanation: Jenkins builds the `Dockerfile`. To drastically speed up consecutive builds, we utilize **Docker BuildKit** and specify `--mount=type=cache` layers for both `apt-get` package installations and `CMake` build objects. This ensures that unchanged dependencies are instantly reused rather than re-downloaded during every pipeline run.

Crucially, this Dockerfile packages both our compiled calculator and a utility named **Gotty**. Gotty captures the backend Linux terminal process and serves it over HTTP

WebSockets, entirely removing the need for end-users to use SSH to access the CLI application.

3.5 Stage 5: Push Docker Image

```
docker push shreyasg28/sci-calc:latest
```

Explanation: Jenkins authenticates and pushes the successfully tested immutable container image globally to the Docker Hub registry for reliable distribution.

3.6 Stage 6: Deploy (Ansible)

```
ansible-playbook -i ansible/inventory.ini ansible/deploy.yml --
    connection=local
```

Explanation: The final stage executes an Ansible playbook to manage the target server state. Ansible gracefully stops and removes any older running versions of the calculator, pulls the fresh image from Docker Hub, and spins up the new web-accessible container mapping host port 8081.

3.7 Post-Build Actions

Regardless of pipeline success or failure, a declarative post-action attempts to dispatch an email notification directly to the developer referencing the current build status and console log URL.

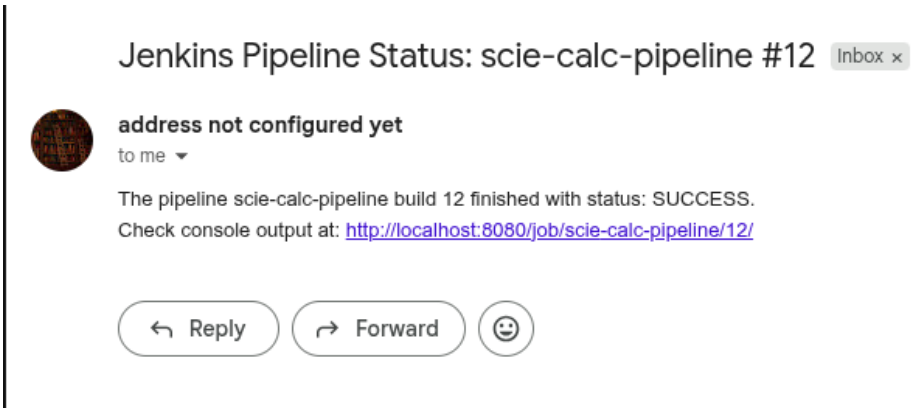

Figure 2: Email Notification dispatched by Jenkins indicating build result.

Digest	Tags	Media type	OS/ARCH	Size	Last pushed	Last pulled	Status
sha256:81335e3adf8e	latest	Image	linux/amd64	159.6 MB	28 minutes	less than 1 day	Active
sha256:579b18dd27e4		Image	linux/amd64	159.6 MB	about 2 hours	less than 1 day	Active
sha256:13a83b1c450c		Image	linux/amd64	159.6 MB	about 2 hours	less than 1 day	Active
sha256:61ca9db08b15		Image	linux/amd64	159.6 MB	about 2 hours	less than 1 day	Active
sha256:86e97c65de84		Image	linux/amd64	156.7 MB	about 3 hours	less than 1 day	Active
sha256:66690b3e6f56		Image	linux/amd64	156.7 MB	about 3 hours	less than 1 day	Active
sha256:fc93f62ac81d		Image	linux/amd64	156.7 MB	about 3 hours	less than 1 day	Active
sha256:a4b9493ab361		Image	linux/amd64	156.7 MB	about 4 hours	less than 1 day	Active
sha256:3c8b556f5b3a		Image	linux/amd64	156.7 MB	about 4 hours	less than 1 day	Active
sha256:f3858e36b208		Image	linux/amd64	156.7 MB	about 4 hours	less than 1 day	Active
sha256:660fb58294ec		Image	linux/amd64	156.7 MB	about 19 hours	less than 1 day	Active
sha256:43b9755ab317		Image	linux/amd64	207.5 MB	about 19 hours	less than 1 day	Active
sha256:806933a5525a		Image	linux/amd64	205.2 MB	about 20 hours	less than 1 day	Active
sha256:7a8d9e5fb673		Image	linux/amd64	201.9 MB	about 24 hours	less than 1 day	Active
sha256:4f0d16042d6e		Image	linux/amd64	201.2 MB	1 day	1 day	Active

Figure 3: Published Docker Images hosted at Docker Hub natively tracking commits.

4 Pipeline Execution and Observability

The following screenshots outline the successful execution of the Jenkins architecture across iterative DevOps branches.

Figure 4: Jenkins Stage View highlighting the explicit pipeline steps and their execution times.

Figure 5: Jenkins Pipeline Execution completing safely with all stages green. This proves the codebase successfully compiled, passed all assertions, packaged into Docker, and deployed via Ansible sequentially without error.

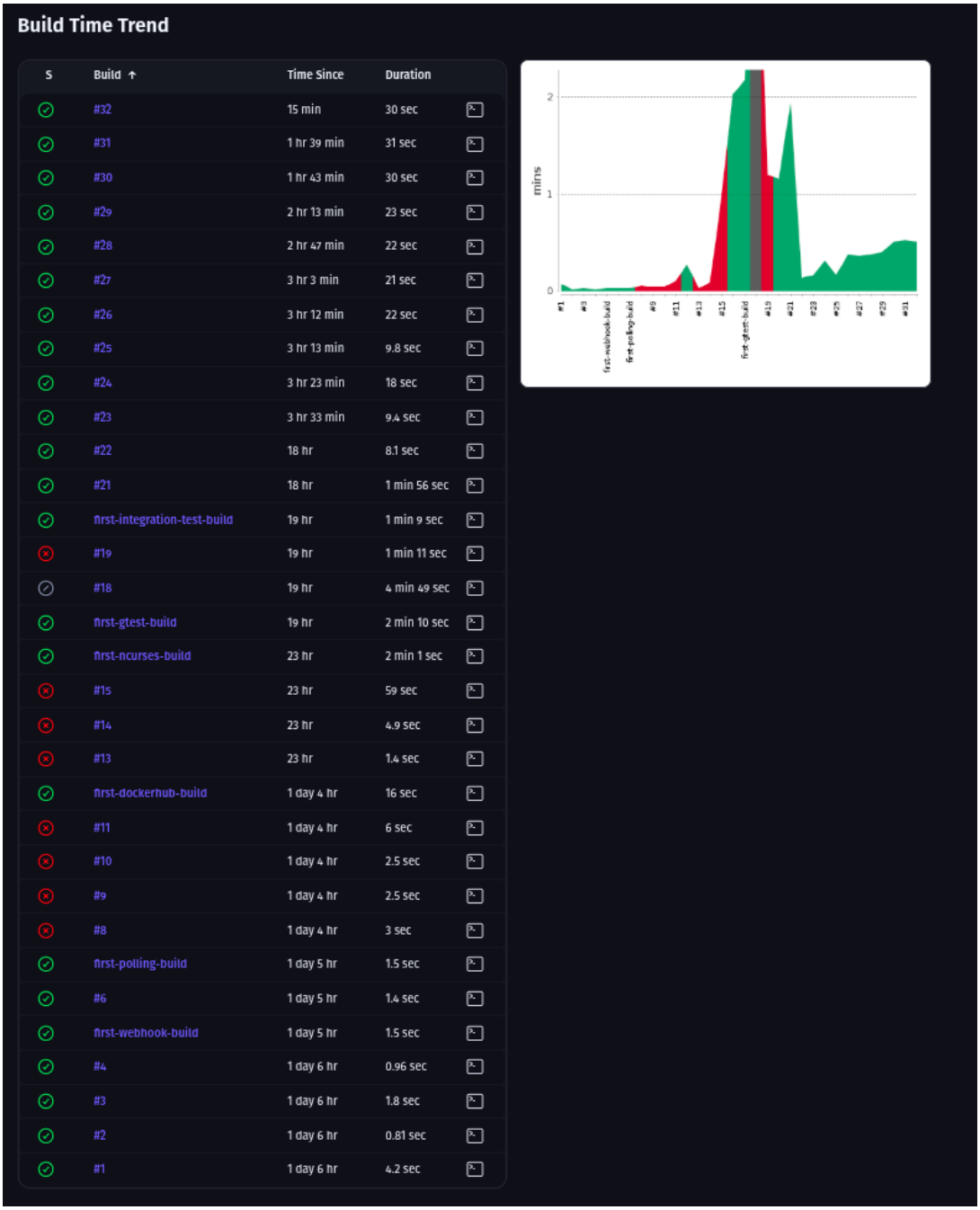

Figure 6: Build Time Trend and Execution History in Jenkins tracking historical failure rates during initial test implementations natively resolving to rapid successful durations. Notice the sharp decline in build duration; this represents the implementation of Docker BuildKit caching mechanisms dropping the entire CI/CD pipeline execution length down to under 30 seconds.

Figure 7: Git Branch Visualization tracking chronological commits mapping feature additions (Factorial, Natural Log, Power functions) and the incremental building of the CI/CD pipeline infrastructure leading to the remote deployment.

5 Test Results and Observations

The pipeline executes dual-layer testing protocols ensuring absolute reliability before touching a production deployment.

5.1 Unit Test Results (Google Test)

Google Test successfully evaluated the ‘src/calculator.cpp‘ logic.

Observations:

- **Positive Validations:** The engine accurately calculates standard expected values (e.g., $4^2 = 16$, $\sqrt{16} = 4$, and $\ln(e) = 1$).
- **Negative/Boundary Validations:** The codebase correctly throws explicit exceptions `std::invalid_argument`, preventing systematic crashes. The tests successfully proved that calculating the square root of a negative number, the factorial of a negative integer, or the natural logarithm of 0 are all safely caught and handled by our model structure rather than crashing the interface.

5.2 Integration Test Results (Expect Automation)

The Expect shell scripting natively simulated end-user keystrokes launching the compiled binary.

Observations:

- Navigational bounds were tested (i.e., pressing "Up/Down" to cycle through Square Root, Factorial, Natural Log, and Power menus).
- Validated that when a user types a mathematically illegal argument directly into the user interface, the `ncurses` terminal properly halts, alerts the user to "Press any key to continue", and safely routes the user back to the Main Menu rather than exiting the process.

6 Application Showcase

Below denotes the application serving gracefully over the target browser window utilizing Web Sockets (Gotty). The user interface is driven effectively utilizing C++ NCurses parsing arrow-key iterations accurately.

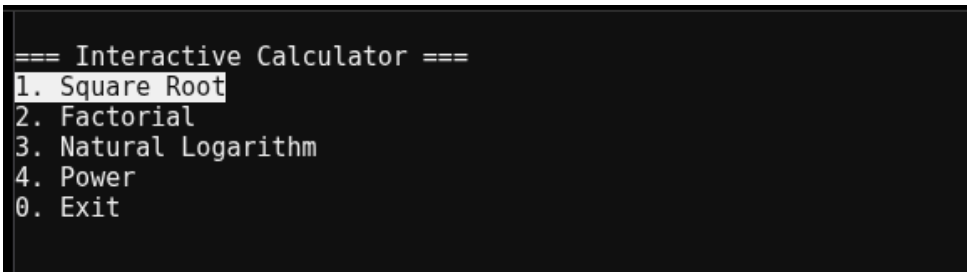

Figure 8: Application Main Menu (Ncurses interface rendered via WebTTY).

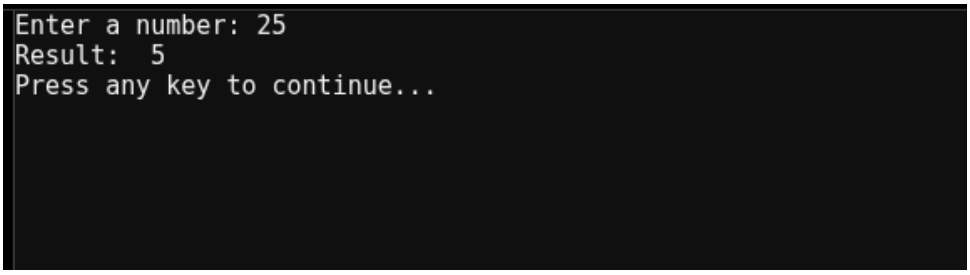

Figure 9: Executing a Square Root calculation spanning valid positive numbers and floating points.

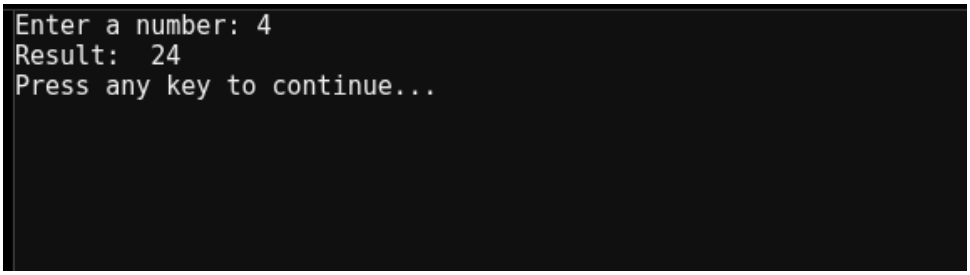

Figure 10: Executing a Factorial calculation over the deployment infrastructure.

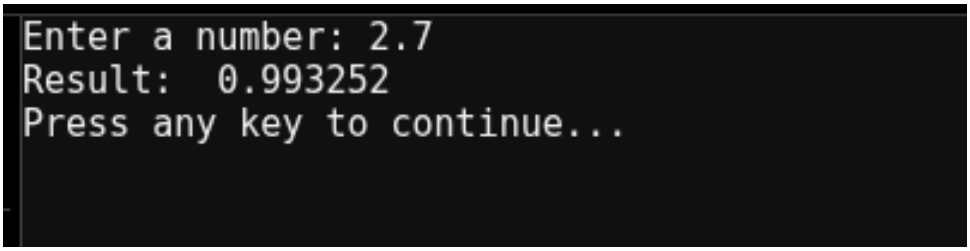

Figure 11: Executing a Natural Logarithm evaluation seamlessly via the Web Terminal.

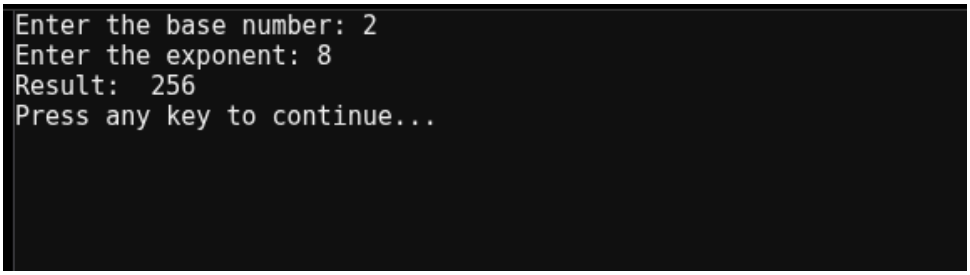

Figure 12: Executing an Exponential Power calculation natively over the live Gotty feed.

7 Links and References

- **GitHub Source Code Repository:** `https://github.com/shreyasg-git/spe-mini-scie-calc.git`
- **Docker Hub Container Registry:** `https://hub.docker.com/r/shreyasg28/sci-calc`